Brandon Gilberto Imanol Retana Fuentes - AL07175444
Máster en Inteligencia Artificial
Gestión de Proyectos de Inteligencia Artificial
Módulo 2

Fase 2. Ejecución - "ClaimCheck AI"

"Documento de validación y pruebas: funcionales, casos extremos, contenedor y rendimiento"

Resultado en una línea por nivel

- **Pruebas automatizadas (pytest):** 84/84 aprobadas, ejecutadas dentro del build de Docker (39 de reglas, 7 del modelo y 38 de la API).
- **Pruebas de punta a punta contra el contenedor:** 50/50 aprobadas en local y 50/50 con los recursos del plan Free de Render (0.1 CPU · 512 MB).
- **Pruebas del contenedor:** 8/8 (usuario sin privilegios, integridad del modelo, variable PORT de la plataforma, Docker Compose).
- **Modelo:** macro-F1 0.852 en 169 claims de prueba frente a 0.599 del mejor zero-shot; cumple el umbral de 0.80. El recall de SALUD (0.857) **no** alcanza 0.90.
- **Conclusión:** el servicio es apto para un piloto con revisión humana obligatoria; no es apto para enrutar sin confirmación hasta mejorar el recall de SALUD.

1. Objetivo y alcance

Este documento demuestra que la solución de la Fase 2 funciona como se especificó y documenta cómo se comporta el modelo, incluidos sus errores. Cubre cinco niveles: el modelo como artefacto, las reglas, la API, el contenedor y el servicio completo en condiciones parecidas a las de la nube. Cada resultado proviene de un archivo del repositorio (`resultados/`), así que cualquier persona puede regenerarlo.

2. Estrategia de pruebas

Nivel	Herramienta	Qué valida	Pruebas	Cuándo corre
Reglas	pytest · `tests/test_reglas.py`	Detección de cada claim regulado y límites exactos (≤, ≥, sólido/líquido, mg→g, % de energía)	39	En cada build
Modelo	pytest · `tests/test_modelo.py`	Integridad SHA-256, contrato de salida, determinismo y regresión del macro-F1	7	En cada build
API	pytest + TestClient · `tests/test_api.py`	Endpoints, validación de entradas, códigos HTTP, frontend servido	38	En cada build
Contenedor	Docker CLI	Usuario, permisos, integridad al arrancar, PORT, healthcheck, Compose	8	Antes de publicar
Punta a punta	`scripts/validar_contenedor.py`	Disponibilidad, funcionales, reproducibilidad, casos extremos, rendimiento	50	CI y después de desplegar

Tabla 1. Niveles de prueba, herramientas y número de pruebas.

Criterios de aceptación (definidos antes de medir; los del modelo son los mismos de la Actividad 4): macro-F1 ≥ 0.80, recall de SALUD ≥ 0.90, exactitud ≥ 90 % en lo que se enruta sin confirmación, latencia p95 ≤ 300 ms, ninguna entrada inválida debe producir un error 500 y las predicciones del contenedor deben coincidir con las del entrenamiento.

Entorno de pruebas

Elemento	Detalle
Imagen	`claimcheck-api:1.0.0`, construida con el Dockerfile del repositorio (Python 3.12, dependencias fijadas)
Motor de contenedores	Docker Engine 29.9.2 sobre Linux x86_64
Escenario local	`--memory 512m --cpus 0.5` (los límites de `docker-compose.yml`)
Escenario plan Free	`--memory 512m --cpus 0.1`, los recursos del plan gratuito de Render
Cliente	`scripts/validar_contenedor.py` con la biblioteca estándar de Python, en el mismo equipo
Datos	`data/claims_etiquetado.csv` (240 claims; partición `test` de 169 para medir)

Tabla 2. Entorno de pruebas.

Para reproducir todo en Windows o macOS basta con seguir `INSTRUCCIONES_DOCKER.md` y correr el script; las predicciones son deterministas y solo cambian los tiempos, que dependen del equipo.

3. Validación del modelo

3.1 Protocolo

1. **Entrenamiento:** los 71 claims de la partición `dev`. El conjunto `test` no se usó para ninguna decisión.
2. **Hiperparámetro C:** validación cruzada estratificada de 5 pliegues en dev. Macro-F1: C=1 → 0.778; C=5, 10 y 20 → 0.759. Se eligió C=1.
3. **Umbral τ:** el menor valor con exactitud ≥ 90 % en lo que se enruta sin confirmar, calculado con probabilidades fuera de pliegue en dev: τ = 0.35.
4. **Medición:** una sola evaluación en los 169 claims de test, el mismo conjunto de la Actividad 4.

3.2 Resultados frente a la línea base zero-shot

Modelo	Tipo	Macro-F1	Accuracy	Recall SALUD	Latencia en CPU
TF-IDF + regresión logística	Supervisado (Fase 2)	**0.852**	0.852	0.857	0.15 ms/claim
mDeBERTa-base	Zero-shot (Act. 4)	0.599	0.610	0.857	10642 ms/claim
XLM-R-large	Zero-shot (Act. 4)	0.592	0.615	0.286	1810 ms/claim
MiniLM-L6	Zero-shot (Act. 4)	0.537	0.550	0.809	377 ms/claim

Tabla 3. Comparación con los modelos zero-shot de la Actividad 4 (conjunto de prueba, 169 claims).

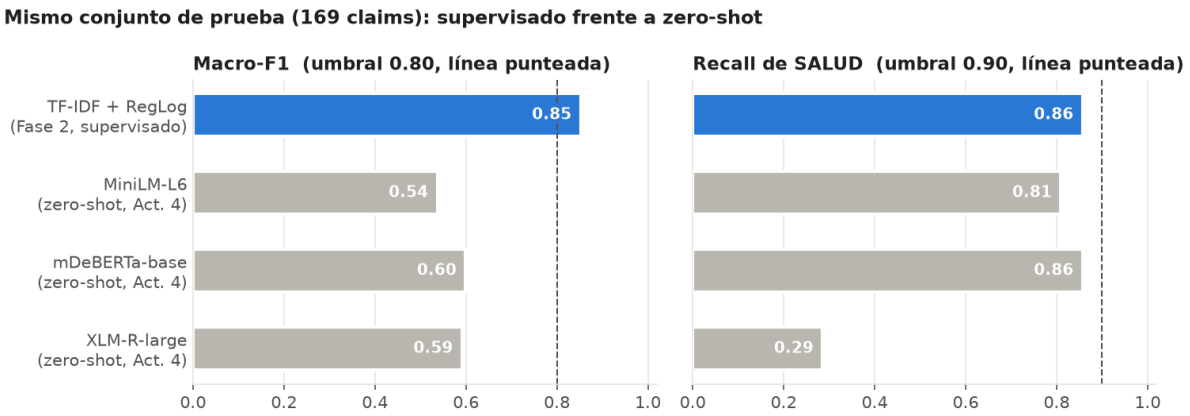


Figura 1. Macro-F1 y recall de SALUD en el mismo conjunto de prueba.

Con solo 71 claims etiquetados, el modelo supervisado supera en 0.25 de macro-F1 al mejor zero-shot y es varios órdenes de magnitud más rápido en CPU (la latencia en CPU de la Actividad 4 se midió en Colab; la de la Fase 2, en el contenedor). Esto confirma la recomendación de la Actividad 4: los modelos zero-shot servían como línea base y el siguiente paso era entrenar con datos propios. El recall de SALUD empata con el de mDeBERTa (0.857) y sigue debajo del umbral.

3.3 Desempeño por clase y casos frontera

Clase	Precisión	Recall	F1	Soporte
AUSENCIA	0.881	0.949	0.914	39
MARKETING	0.816	0.795	0.805	39
NUTRIMENTAL	0.851	0.816	0.833	49
SALUD	0.857	0.857	0.857	42

Tabla 4. Métricas por clase en el conjunto de prueba.

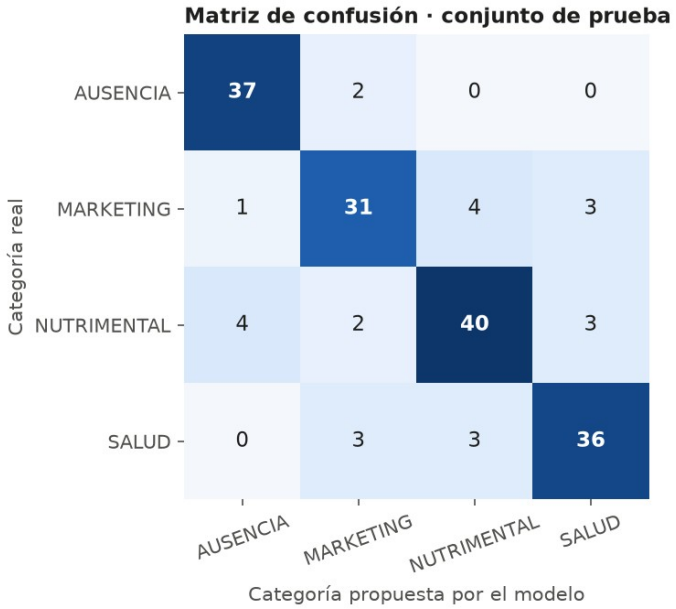


Figura 2. Matriz de confusión en el conjunto de prueba.

AUSENCIA es la clase más estable (recall 0.949). Las confusiones se concentran entre SALUD y NUTRIMENTAL y entre MARKETING y las demás. En los 29 casos frontera la accuracy baja a 0.690, frente a 0.886 en los casos estándar: el modelo se equivoca donde también dudaría una persona.

3.4 Umbral de confianza y verificación cruzada

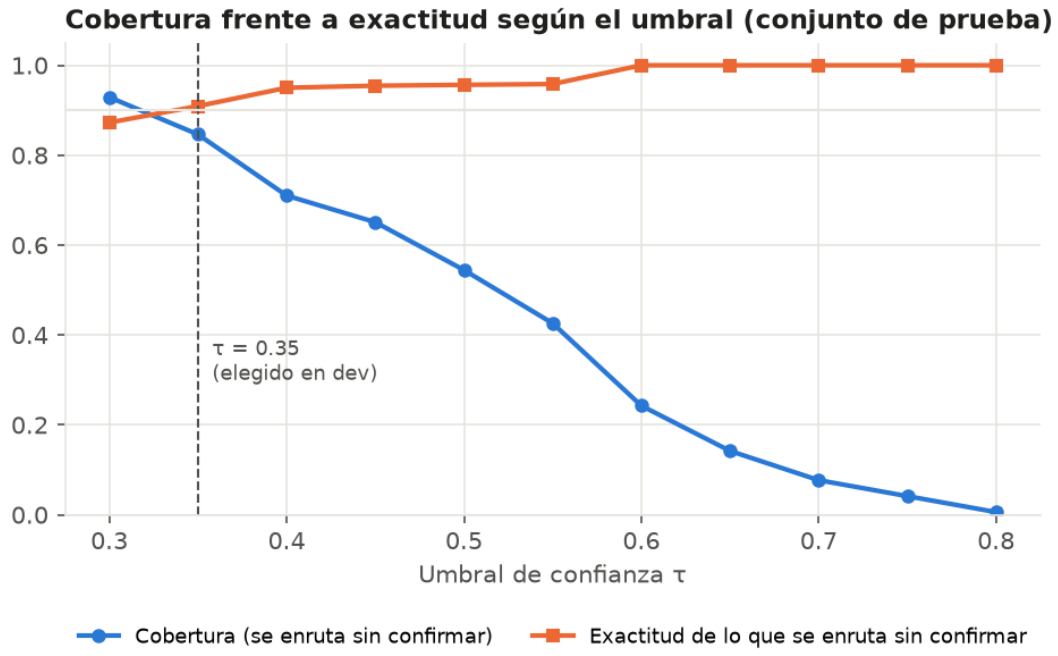


Figura 3. Cobertura y exactitud de lo que se enruta sin confirmar, según τ.

Con τ = 0.35 el modelo enruta sin confirmación el 84.6 % de los claims con exactitud de 90.9 %; en el resto, la exactitud cae a 53.8 %, lo que confirma que la confianza separa bien los casos dudosos. La API agrega una segunda señal: si una regla reconoce un claim nutrimental y el modelo propone otra clase, también lo marca. Con ambas señales:

Indicador del sistema (API, 169 claims de prueba)	Valor
Claims marcados para confirmar	30 (17.8 %)
Exactitud de lo que se enruta sin confirmar	92.8 %
Errores del modelo	25
Errores marcados para el revisor	15 (12 por τ, 3 por la regla)
Errores que pasan sin marca	10 (5.9 % de los claims)

Tabla 5. Indicadores del sistema con el umbral τ y la verificación cruzada.

3.5 Análisis de errores

Los errores que pasan sin marca son el riesgo real, porque el revisor no recibe aviso. Son estos:

Claim	Real	Propuesta	Confianza
0 % conservadores, 100 % sabor	AUSENCIA	MARKETING	0.36
Listo en 3 minutos	MARKETING	NUTRIMENTAL	0.36
Apoya tu energía durante todo el día	SALUD	MARKETING	0.42
Una dieta baja en grasas saturadas y colesterol puede reducir el riesgo de enfermedad cardiaca	SALUD	NUTRIMENTAL	0.39
Una porción cubre el 50 % de tu requerimiento diario de vitamina C	NUTRIMENTAL	SALUD	0.36
Del campo a tu mesa	MARKETING	SALUD	0.39
Natural y saludable	SALUD	MARKETING	0.53

Claim	Real	Propuesta	Confianza
La fibra soluble de la avena, como parte de una dieta baja en grasas saturadas, puede reducir el riesgo de enfermedad coronaria	SALUD	NUTRIMENTAL	0.36
Sin calorías	NUTRIMENTAL	AUSENCIA	0.58
Fuente natural de fibra	NUTRIMENTAL	MARKETING	0.37

Tabla 6. Errores del modelo que pasan sin marca para el revisor.

- **Reducción de riesgo con nutrimentos** ("Una dieta baja en grasas saturadas… puede reducir el riesgo…"): el modelo se ancla en las palabras nutrimentales e ignora la relación con la enfermedad. Es el error más costoso, porque una declaración de reducción de riesgo exige autorización específica.
- **Claims de salud genéricos** ("Natural y saludable", "Apoya tu energía durante todo el día") se leen como mercadotecnia. Cuatro de los diez errores silenciosos son claims de SALUD.
- **Frases cortas de origen o conveniencia** ("Del campo a tu mesa", "Listo en 3 minutos") no tienen vocabulario distintivo.
- **"Sin calorías" y "Fuente natural de fibra"** quedan fuera de las 14 reglas tal como están redactadas, así que la verificación cruzada no las detecta. En cambio, "Sin sal añadida" y "Sin azúcares añadidos", mal enrutados a AUSENCIA, sí quedan marcados por su regla.

4. Pruebas funcionales

Disponibilidad y funcionalidad del servicio completo, ejecutadas con `scripts/validar_contenedor.py` contra el contenedor en ejecución.

ID	Prueba	Esperado	Obtenido	Estado
D-01	GET /health: servicio vivo y modelo íntegro	200, estado ok, SHA-256 verificado	200, ok, integridad True	**Aprobada**
D-02	GET /api/v1/info: metadatos del modelo	200 y 4 clases	200, v1.0.0, τ=0.35	**Aprobada**
D-03	GET /: el backend sirve el frontend	200 y HTML de ClaimCheck	200, HTML con título	**Aprobada**
D-04	GET /static/app.js: cliente JavaScript	200	200	**Aprobada**
D-05	GET /openapi.json: contrato de la API publicado	6 endpoints documentados	200, 6/6 endpoints	**Aprobada**
D-06	GET /docs: documentación interactiva (Swagger UI)	200	200	**Aprobada**
F-01	Clasificar «Sin gluten»	200, AUSENCIA	200, AUSENCIA (0.6412)	**Aprobada**
F-02	Clasificar «Receta tradicional»	200, MARKETING	200, MARKETING (0.6907)	**Aprobada**
F-03	Clasificar «Bajo en sodio»	200, NUTRIMENTAL	200, NUTRIMENTAL (0.4979)	**Aprobada**
F-04	Clasificar «El calcio es necesario para el manteni»	200, SALUD	200, SALUD (0.6875)	**Aprobada**
F-05	Lote de 3 claims conserva el orden	200 y mismo orden	200, n=3	**Aprobada**
F-06	Etiqueta sólida (cereal): reglas de fibra, grasa y azúcares	fibra CUMPLE · grasa NO_CUMPLE · azúcares REQUIERE_EVIDENCIA · gluten sin regla	200, fibra CUMPLE · grasa NO_CUMPLE · azúcares REQUIERE_EVIDENCIA · gluten None	**Aprobada**
F-07	Etiqueta líquida: umbrales por 100 ml y % de energía	CUMPLE · CUMPLE · NO_CUMPLE (11.1 % < 12 %)	200, CUMPLE · CUMPLE · NO_CUMPLE	**Aprobada**

ID	Prueba	Esperado	Obtenido	Estado
F-08	Trazabilidad: la API devuelve X-Request-ID y tiempo de proceso	X-Request-ID = val-001	200, val-001	**Aprobada**

Tabla 7. Pruebas de disponibilidad y funcionales contra el contenedor.

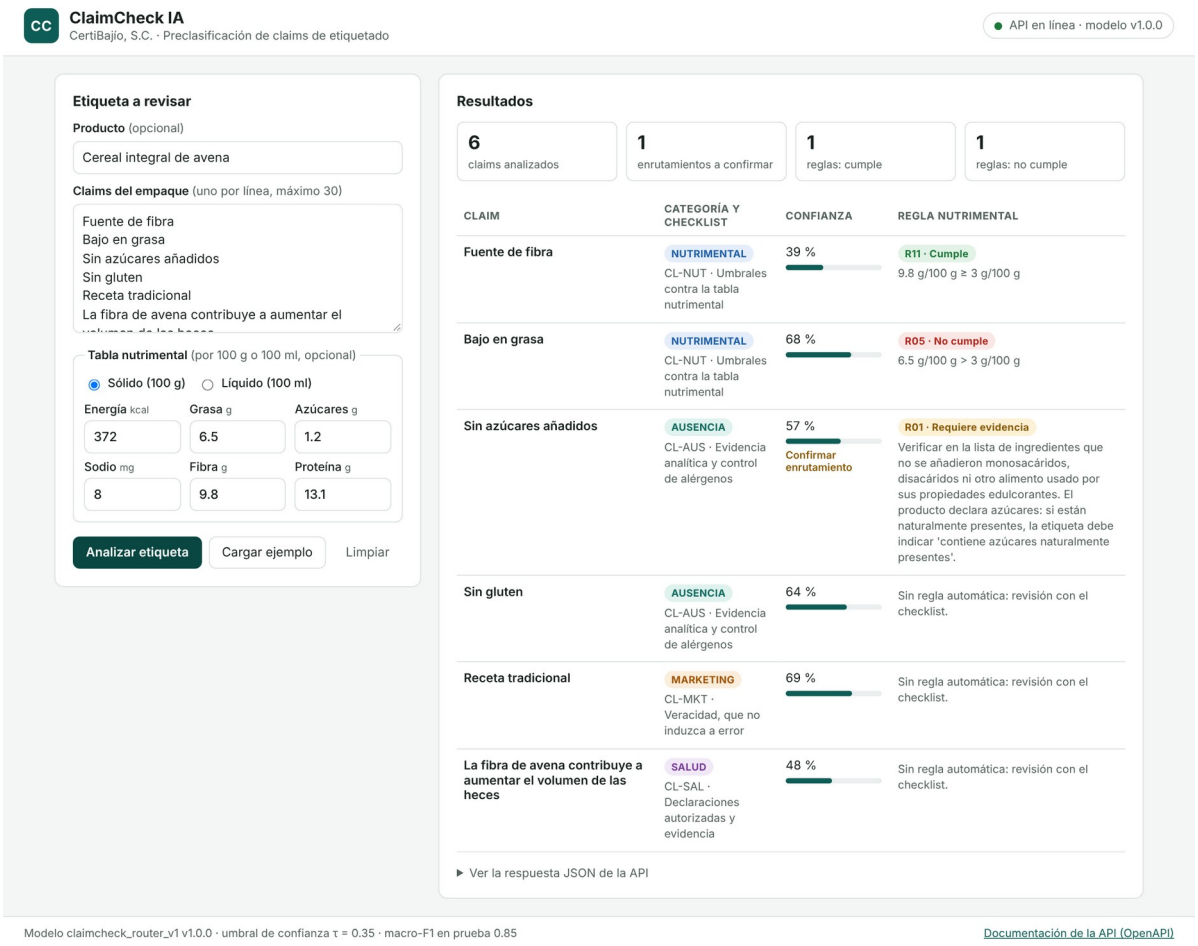


Figura 4. Frontend consumiendo la API: enrutamiento, confianza, veredicto de regla y claim marcado para confirmar.

5. Casos extremos

El objetivo es que ninguna entrada, por anómala que sea, provoque un error 500 o una respuesta engañosa. Las entradas inválidas deben rechazarse con un código 4xx y un mensaje claro; las válidas pero atípicas deben responderse y, de preferencia, marcarse para el revisor.

ID	Caso	Esperado	Obtenido	Estado
E-01	Texto vacío	422 (validación)	422	**Aprobada**
E-02	Solo espacios	422 (validación)	422	**Aprobada**
E-03	Un carácter (mínimo 2)	422 (validación)	422	**Aprobada**
E-04	301 caracteres (máximo 300)	422 (validación)	422	**Aprobada**
E-05	Tipo incorrecto (número)	422 (validación)	422	**Aprobada**
E-06	Campo obligatorio ausente	422 (validación)	422	**Aprobada**
E-07	Campo no permitido	422 (validación)	422	**Aprobada**

ID	Caso	Esperado	Obtenido	Estado
E-08	Exactamente 300 caracteres (límite)	200	200	**Aprobada**
E-09	JSON malformado	422	422	**Aprobada**
E-10	Content-Type text/plain en lugar de JSON	422	422	**Aprobada**
E-11	Lote vacío	422	422	**Aprobada**
E-12	Lote de 51 claims (máximo 50)	422	422	**Aprobada**
E-13	Lote de 50 claims (límite)	200	200	**Aprobada**
E-14	Cuerpo de 70 KB (máximo 64 KB)	413	413	**Aprobada**
E-15	Nutrimento negativo (grasa -1 g)	422	422	**Aprobada**
E-16	Sodio fuera de rango (50 000 mg)	422	422	**Aprobada**
E-17	Masa imposible (grasa 80 g + proteína 40 g)	422	422	**Aprobada**
E-18	Estado físico inválido	422	422	**Aprobada**
E-19	Límite de regla: «Bajo en grasa» con 3.0 g/100 g	CUMPLE	200, CUMPLE	**Aprobada**
E-20	Límite de regla: «Bajo en grasa» con 3.01 g/100 g	NO_CUMPLE	200, NO_CUMPLE	**Aprobada**
E-21	Energía 0 kcal en regla de proteína (división entre cero)	200, DATOS_INSUFICIENTES	200, DATOS_INSUFICIENTES	**Aprobada**
E-22	Mayúsculas, sin acentos y Unicode descompuesto (NFD)	Misma predicción en las 3 variantes	idénticas: AUSENCIA	**Aprobada**
E-23	Solo emojis	200, texto devuelto sin interpretar (sin error 500)	200, MARKETING conf 0.3021, a confirmar	**Aprobada**
E-24	Solo números	200, texto devuelto sin interpretar (sin error 500)	200, MARKETING conf 0.3258, a confirmar	**Aprobada**
E-25	Inyección HTML/JS	200, texto devuelto sin interpretar (sin error 500)	200, MARKETING conf 0.3119, a confirmar	**Aprobada**
E-26	Inyección SQL	200, texto devuelto sin interpretar (sin error 500)	200, MARKETING conf 0.3258, a confirmar	**Aprobada**
E-27	Claim en inglés (fuera del alcance)	200, texto devuelto sin interpretar (sin error 500)	200, AUSENCIA conf 0.4565, sin marca	**Aprobada**
E-28	Texto fuera del dominio	200, texto devuelto sin interpretar (sin error 500)	200, SALUD conf 0.3099, a confirmar	**Aprobada**
E-29	Ruta inexistente	404	404	**Aprobada**
E-30	Método no permitido (GET en endpoint POST)	405	405	**Aprobada**

Tabla 8. Casos extremos evaluados.

Observaciones. Los textos sin sentido (emojis, números, inyecciones) reciben confianzas de 0.30 a 0.33, por debajo de τ, y quedan marcados: el umbral funciona como detector de entradas fuera de dominio. "Gluten free" se enruta bien a AUSENCIA gracias a los n-gramas de caracteres, aunque el inglés está fuera del alcance. La inyección HTML se devuelve como texto y el frontend la muestra con `textContent`, sin ejecutarla.

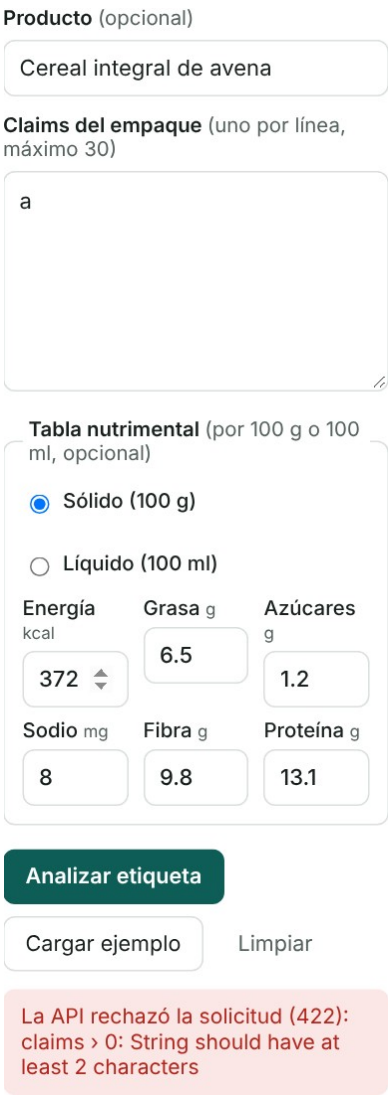


Figura 5. La interfaz muestra el rechazo 422 de la API.

6. Pruebas del contenedor

ID	Prueba	Resultado
C-01	Usuario de ejecución	claimcheck (uid 10001)
C-02	Escritura sobre el código como usuario de la app	touch: cannot touch 'app/prueba.txt': Permission denied
C-03	Escritura sobre el modelo	sh: 1: cannot create modelos/claimcheck_router_v1.joblib: Permission denied
C-04	Modelo alterado montado en el contenedor	estado: exited (exit 3); log: ErrorIntegridad: SHA-256 del modelo no coincide (esperado 3b96cfeffab2..., obtenido ffd0b0ac68b5...)
C-05	PORT=10000 (como Render)	{"estado":"ok","version_api":"1.0.0","version_modelo":"1.0.0","modelo_cargado":true,"integridad_verificada":true}; log: Uvicorn running on http://0.0.0.0:10000
C-06	HEALTHCHECK con PORT=10000	healthy
C-07	docker compose up -d	claimcheck-api Up 6 seconds (healthy); límites: Memory=536870912 NanoCpus=500000000
C-08	Respuesta vía compose	AUSENCIA 0.6412 CL-AUS · Evidencia analítica y control de alérgenos

Tabla 9. Pruebas del contenedor.

Compuerta de calidad en el build. La etapa final copia un archivo que solo existe si las 84 pruebas pasan; el log de construcción lo confirma:

```
#16 [test 8/8] RUN python -m pytest -q && touch /tmp/pruebas_superadas
#16 2.414 84 passed in 0.61s
#17 [runtime 1/5] COPY --from=test /tmp/pruebas_superadas /srv/claimcheck/.pruebas_superadas
#22 naming to docker.io/library/claimcheck-api:1.0.0 done
```

Arranque y salud:

```
INFO:     Started server process [1]
INFO:     Waiting for application startup.
2026-10-30 03:01:54,652 INFO claimcheck.modelo | Modelo 1.0.0 cargado (4 clases, τ=0.35, SHA-256
verificado)
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     172.17.0.1:59204 - "GET /health HTTP/1.1" 200 OK

NAMES             IMAGE                  STATUS                         PORTS
claimcheck-free   claimcheck-api:1.0.0   Up 41 seconds (healthy)        0.0.0.0:8001->8000/tcp
claimcheck        claimcheck-api:1.0.0   Up About a minute (healthy)    0.0.0.0:8000->8000/tcp
```

7. Reproducibilidad

ID	Prueba	Esperado	Obtenido	Estado
R-01	Las 169 predicciones del test vía API = fuera de línea	169/169 iguales	169/169 iguales, Δconfianza máx 0.0e+00	**Aprobada**
R-02	Macro-F1 recalculado con las respuestas de la API	= 0.8523 (metadatos)	0.8523	**Aprobada**
R-03	Errores del modelo que el sistema marca para el revisor	Métrica (se reporta)	15/25 errores marcados (τ 12, regla 3)	**Aprobada**

Tabla 10. Pruebas de reproducibilidad.

Las 169 predicciones obtenidas a través de la API del contenedor son idénticas, con la misma confianza, a las del entrenamiento fuera de línea. El modelo se entrenó dentro del mismo entorno Python 3.12 de la imagen, con las versiones fijadas, y su huella SHA-256 se verifica al arrancar: lo que se validó es exactamente lo que se despliega.

8. Rendimiento

Indicador	Local (0.5 CPU · 512 MB)	Plan Free simulado (0.1 CPU · 512 MB)
Tiempo hasta responder /health	3.2 s	19.2 s
Memoria en uso	99 MB	99 MB
Latencia secuencial (200 y 100 solicitudes) p50 / p95	4.9 / 44.3 ms	13.1 / 95.3 ms
Concurrencia	400 solicitudes, 16 hilos: 128/s, 0 errores	200 solicitudes, 8 hilos: 22/s, 0 errores
p95 bajo concurrencia	178 ms	495 ms
Lote de 50 claims (mediana)	7.7 ms (0.15 ms/claim)	96.7 ms (1.93 ms/claim)

Tabla 11. Rendimiento en los dos escenarios de recursos.

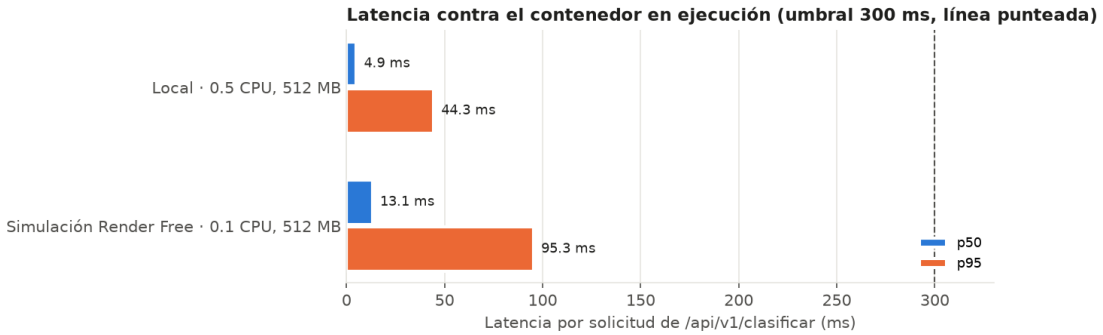


Figura 6. Latencia por solicitud frente al umbral de 300 ms.

En ambos escenarios el p95 queda muy por debajo de 300 ms. Con 0.1 CPU la diferencia entre p50 y p95 viene de la cuota de CPU del contenedor, no del modelo. El volumen anual del caso de negocio (~86 400 claims, unos 240 al día) no se acerca a la capacidad medida. El cuello de botella en la nube será el arranque en frío del plan Free (cerca de un minuto tras 15 minutos sin tráfico), no la inferencia.

9. Resultados y conclusiones

Criterio de aceptación	Resultado	¿Cumple?
Macro-F1 ≥ 0.80 (test)	0.852	**Sí**
Recall de SALUD ≥ 0.90 (test)	0.857	**No**
Exactitud ≥ 90 % en lo que se enruta sin confirmar	92.8 %	**Sí**
Latencia p95 ≤ 300 ms	44 ms local · 95 ms plan Free	**Sí**
Sin errores 500 ante entradas anómalas	30/30 casos extremos	**Sí**
Contenedor = entrenamiento	169/169 predicciones idénticas	**Sí**
Cabe en el plan Free (512 MB)	99 MB en uso	**Sí**

Tabla 12. Criterios de aceptación y resultado.

Conclusiones

- **La solución es técnicamente desplegable.** La imagen se construye solo si pasan las pruebas, arranca en segundos, verifica su modelo, responde dentro del umbral incluso con 0.1 CPU y rechaza de forma controlada todas las entradas anómalas probadas.
- **El modelo mejora mucho la línea base, pero no es autónomo.** Enruta bien el 85 % de los claims y marca los dudosos, pero 10 de 169 errores pasan sin aviso y 4 de ellos son claims de SALUD. Por eso la confirmación del revisor sigue siendo obligatoria, como se planteó desde la Fase 1.
- **Las métricas son un techo optimista.** El dataset es propio, limpio y pequeño; con etiquetas reales (OCR, claims combinados, inglés) el desempeño será menor.

Acciones siguientes

1. Etiquetar al menos 200 claims reales de SALUD, con énfasis en reducción de riesgo y beneficios genéricos, y reentrenar; la meta es recall ≥ 0.90.
2. Agregar una verificación cruzada de salud, análoga a la nutrimental: si el claim contiene expresiones como "reducir el riesgo", "contribuye a" o "ayuda a" y el modelo no propone SALUD, marcarlo.

3. Ampliar las reglas a "sin calorías" y a variantes con calificativos ("fuente natural de fibra"), y añadir criterios de NOM-051 y 21 CFR 101.
4. Registrar cada confirmación o corrección del revisor como nueva etiqueta para monitorear la deriva y alimentar el reentrenamiento.
5. Antes de usar etiquetas de clientes: autenticación, límite de tasa y despliegue híbrido (manual de despliegue, sección 6).

10. Referencias

- Comisión Europea. (2006). Reglamento (CE) n.º 1924/2006 relativo a las declaraciones nutricionales y de propiedades saludables en los alimentos. *Diario Oficial de la Unión Europea*, L 404.
- Docker Inc. (s.f.). *Multi-stage builds*. https://docs.docker.com/build/building/multi-stage/
- FastAPI. (s.f.). *Testing*. https://fastapi.tiangolo.com/tutorial/testing/
- International Organization for Standardization. (2022). *ISO/IEC 23053:2022 Framework for artificial intelligence (AI) systems using machine learning (ML)*. ISO.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.
- Render. (s.f.). *Deploy for free*. https://render.com/docs/free

11. Anexo. Cómo reproducir las pruebas

```
docker build -t claimcheck-api:1.0.0 .                                  # 84 pruebas dentro del
build
docker run -d --name claimcheck -p 8000:8000 --memory 512m --cpus 0.5 claimcheck-api:1.0.0
python scripts/validar_contenedor.py --etiqueta local                    # 50 pruebas e2e
docker run -d --name claimcheck-free -p 8001:8000 --memory 512m --cpus 0.1 claimcheck-api:1.0.0
python scripts/validar_contenedor.py --url http://localhost:8001 --etiqueta
simulacion_render_free \
  --n-latencia 100 --n-concurrencia 200 --hilos 8
pip install -r requirements-analisis.txt && python -m src.evaluar_modelo   # figuras
```

Archivos de evidencia en el repositorio:

- `resultados/validacion_local.json` y `resultados/validacion_simulacion_render_free.json`: las 50 pruebas de punta a punta con esperado, obtenido y estado.
- `resultados/predicciones_test.csv` y `resultados/metricas_modelo.json`: predicciones y métricas del modelo.
- `resultados/logs/`: build, arranque, `docker ps`, `docker stats` y pruebas del contenedor.
- `docs/evidencias/`: capturas del frontend consumiendo la API.